



Universidade Estadual Paulista “Júlio de Mesquita Filho” (UNESP)
Faculdade de Ciências – Bauru
Departamento de Computação

Sistema de Acompanhamento e Avaliação de Séries de TV

Modelagem conceitual, lógica e física de um banco de dados
relacional

Banco de Dados I

Igor dos Reis Gomes – RA: 241025265

Nome do segundo integrante – RA: 000000000

Nome do terceiro integrante – RA: 000000000

Nome do quarto integrante – RA: 000000000

Bauru – SP
2026

Sumário

Como utilizar este documento	1
1 Modelagem Conceitual	2
1.1 Descrição do minimundo	2
1.2 Regras de negócio	3
1.3 Entidades e atributos	4
1.3.1 <code>usuario</code>	5
1.3.2 <code>serie</code>	5
1.3.3 <code>temporada</code> – entidade fraca	5
1.3.4 <code>episodio</code> – entidade fraca	5
1.3.5 <code>genero</code>	6
1.3.6 <code>pessoa</code>	6
1.3.7 <code>avaliacao</code> – entidade genérica	6
1.3.8 <code>avaliacao_serie</code> , <code>avaliacao_temporada</code> , <code>avaliacao_episodio</code>	6
1.3.9 <code>comentario</code>	6
1.4 Relacionamentos e cardinalidades	7
1.5 Generalização e especialização	7
1.6 Agregação	8
1.7 Atributos derivados	8
1.8 Diagrama entidade-relacionamento	8
2 Modelagem Lógica	10
2.1 Regras de mapeamento aplicadas	10
2.1.1 Tratamento da especialização	10
2.1.2 Tratamento das entidades fracas	11
2.2 Esquema relacional	11
2.3 Dependências funcionais	11
2.4 Normalização	12
2.5 Diagrama lógico	13
3 Modelagem Física	14
3.1 Convenções de nomenclatura e criação do esquema	14
3.2 Definição de dados (DDL)	14
3.3 Restrições de integridade	14
3.4 Gatilhos	15
3.5 Visões	16
3.6 Carga de dados (DML)	17
3.7 Consultas de exemplo	17
3.7.1 Consulta 1 – Ranking de séries por nota média	17
3.7.2 Consulta 2 – Atividade recente de quem o usuário segue	17
3.7.3 Consulta 3 – Séries na lista de interesse ainda não iniciadas	17

3.8 Rastreabilidade das regras de negócio 18

A Divisão de trabalho e convenções do grupo 19

A.1 Atribuições 19

A.2 Contrato de nomes 19

A.3 Verificação antes da entrega 19

Como utilizar este documento

Este arquivo já contém a **modelagem conceitual completa** (Capítulo 1). Os Capítulos 2 e 3 estão estruturados, mas precisam ser escritos pelos demais integrantes.

O Capítulo 1 não é apenas a primeira etapa do trabalho: ele **fixa o vocabulário** do projeto. Os nomes de entidades, atributos, tabelas e colunas definidos ali valem para todo o restante do documento e para o script SQL. Se alguma etapa posterior precisar alterar um nome, a alteração deve ser feita *também* no Capítulo 1, para que o documento não se contradiga – um esquema físico que não corresponde ao diagrama conceitual é o erro mais comum e mais caro deste tipo de trabalho.

Os trechos marcados como *Orientação* explicam o que se espera em cada seção. Eles podem ser ocultados alterando-se, no início do arquivo, `\orientacoestrue` para `\orientacoesfalse`. O Apêndice de divisão de trabalho também desaparece nesse caso, pois é material interno do grupo.

Capítulo 1

Modelagem Conceitual

Este capítulo descreve o minimundo escolhido pelo grupo, enuncia as regras de negócio que o banco de dados deve sustentar e apresenta o diagrama entidade-relacionamento (DER) correspondente.

O domínio foi escolhido por concentrar, em um cenário familiar a qualquer usuário, as construções que a modelagem conceitual precisa exercitar: uma hierarquia de composição em três níveis, relacionamentos muitos-para-muitos com atributos próprios, um grafo social que relaciona um conjunto de entidades a ele mesmo e um ato – o de avaliar – que incide indistintamente sobre três tipos de alvo. Nenhuma dessas construções foi introduzida artificialmente para satisfazer o enunciado: todas decorrem da forma como aplicações de acompanhamento de séries efetivamente funcionam.

1.1 Descrição do minimundo

O sistema modelado é uma rede social voltada ao acompanhamento e à avaliação de séries de televisão. Ele não exhibe conteúdo audiovisual: registra e organiza o que as pessoas assistem e o que elas acham do que assistiram. O paralelo mais próximo são as plataformas de catalogação de filmes e de livros, como Letterboxd e Goodreads, transposto para o formato seriado – no qual a obra se subdivide em unidades menores, e cada uma dessas unidades pode ser assistida e avaliada separadamente.

O acervo do sistema é composto por **séries**, cada uma organizada em **temporadas**, que por sua vez se dividem em **episódios**. Essa hierarquia de três níveis é a espinha dorsal do minimundo, porque quase toda interação do usuário acontece em um desses três níveis. Cada série é classificada em um ou mais **gêneros** e conta com a participação de **pessoas** do meio audiovisual, que podem atuar como atores, diretores, roteiristas ou criadores. Uma mesma pessoa pode exercer funções diferentes em séries diferentes – e até na mesma série.

Os **usuários** do sistema mantêm uma relação individual com cada série do acervo. Essa relação tem duas faces complementares. A primeira é a **intenção declarada**: o usuário marca a série como algo que pretende assistir, que está assistindo ou que abandonou. A segunda é o **histórico efetivo**: o usuário registra, episódio a episódio, o que de fato assistiu e em que data. As duas faces não são redundantes. Uma série pode estar marcada como “pretendo assistir” sem nenhum episódio registrado, e o progresso real de quem está assistindo só pode ser calculado a partir dos registros individuais.

Sobre qualquer um dos três níveis da hierarquia, o usuário pode publicar uma **avaliação**: uma nota, um texto crítico, ou ambos. Avaliar uma série inteira, uma temporada específica ou um episódio isolado são atos de natureza idêntica – mudam apenas o alvo e o grau de detalhe – e por isso são tratados no modelo como um único conceito especializado em três formas.

A camada social se organiza em torno dessas avaliações. Um usuário pode **seguir** outros

usuários, o que constitui seu acompanhamento de atividade; pode **curtir** avaliações alheias; e pode **comentar** nelas, inclusive respondendo a comentários de outras pessoas. É importante notar que a curtida e o comentário não incidem sobre a série nem sobre o autor isoladamente: eles incidem sobre o *ato de avaliar*, isto é, sobre a associação entre um usuário e um item do acervo.

O minimundo organiza-se, portanto, em três pilares: o **acervo** (o dado sobre a obra), o **acompanhamento** (a relação entre usuário e obra) e o **social** (a relação entre usuários, mediada por avaliações).

1.2 Regras de negócio

***Orientação:** Cada regra abaixo precisa ser rastreável até uma construção concreta do banco. Ao escrever os Capítulos 2 e 3, cite o código da regra (RN01, RN02, ...) ao lado da restrição, do gatilho ou da chave que a implementa. A Seção 3.8 traz a tabela de rastreabilidade a ser preenchida.*

Usuários e relação social

- RN01** Cada usuário é identificado por um **username** único no sistema, composto apenas por letras minúsculas, dígitos e sublinhado, com 3 a 30 caracteres. O endereço de e-mail também é único.
- RN02** Um usuário pode seguir outros usuários. A relação é **unidirecional**: seguir alguém não implica ser seguido de volta.
- RN03** Nenhum usuário pode seguir a si mesmo.
- RN04** A exclusão de um usuário implica a exclusão de todo o seu histórico: estados declarados, registros de exibição, avaliações, curtidas e comentários.

Acervo

- RN05** Toda série pertence a pelo menos um gênero e pode pertencer a vários.
- RN06** Toda temporada pertence a exatamente uma série, e seu número é único *dentro* daquela série. Não existe temporada fora de uma série.
- RN07** Todo episódio pertence a exatamente uma temporada, e seu número é único *dentro* daquela temporada. Não existe episódio fora de uma temporada.
- RN08** A exclusão de uma série implica a exclusão de suas temporadas e, por consequência, de seus episódios.
- RN09** Uma pessoa pode participar de várias séries, e em cada série pode exercer mais de uma função (ator, diretor, roteirista ou criador).
- RN10** O nome do personagem só faz sentido para a função de ator; nas demais funções, esse dado é inexistente.

Acompanhamento

- RN11** Um usuário pode registrar o mesmo episódio mais de uma vez, em datas diferentes. Cada registro é um evento independente, o que permite representar reexibições.
- RN12** A data de um registro de exibição não pode ser futura.
- RN13** Um usuário declara, para cada série, no máximo **um** estado, entre: *watchlist* (pretende assistir), *assistindo* ou *abandonada*. Séries sobre as quais o usuário nada declarou simplesmente não possuem estado registrado.
- RN14** Registrar a exibição de um episódio de uma série implica que o usuário passa a estar *assistindo* aquela série. Se ele havia declarado *watchlist*, o estado é promovido automaticamente.

Avaliações

- RN15** Uma avaliação é publicada por exatamente um usuário e incide sobre exatamente um alvo, que é **ou** uma série, **ou** uma temporada, **ou** um episódio. Não existe avaliação sem alvo, nem avaliação com mais de um alvo.
- RN16** A nota atribuída varia de 0,5 a 5,0, em incrementos de meio ponto. A nota é opcional.
- RN17** Uma avaliação não pode ser inteiramente vazia: deve conter ao menos uma nota ou ao menos um texto.
- RN18** Um usuário só pode avaliar um episódio que ele já tenha registrado como assistido.
- RN19** Um usuário possui no máximo uma avaliação por alvo. Reavaliar significa atualizar a avaliação existente, não criar outra.
- RN20** O autor pode sinalizar que sua avaliação contém spoiler, para que a aplicação oculte o texto por padrão.

Interação sobre avaliações

- RN21** Um usuário pode curtir uma avaliação uma única vez.
- RN22** Nenhum usuário pode curtir a própria avaliação.
- RN23** Uma avaliação pode receber comentários de qualquer usuário. Um comentário pode ser resposta a outro comentário, desde que ambos pertençam à mesma avaliação.
- RN24** A exclusão de uma avaliação implica a exclusão de suas curtidas e de seus comentários.

1.3 Entidades e atributos

O modelo possui **onze conjuntos de entidades**. Os relacionamentos do tipo muitos-para-muitos, que darão origem a tabelas próprias apenas na etapa lógica, estão descritos na Seção 1.4 e não são contados aqui.

Orientação: A notação de chave usada nas tabelas a seguir: **PK** identificador primário; **PP** chave parcial (entidade fraca, única apenas dentro do proprietário); **U** valor único no conjunto; **D** discriminador de especialização. Atributos marcados como obrigatórios não admitem valor nulo.

1.3.1 usuario

Representa a pessoa que utiliza o sistema.

Atributo	Tipo	Chave	Obrig.	Descrição
id_usuario	inteiro	PK	sim	Identificador interno
username	texto(30)	U	sim	Identificador público; minúsculas, dígitos e _
email	texto(120)	U	sim	Endereço de contato
nome_exibicao	texto(60)		não	Nome apresentado na interface
bio	texto		não	Descrição livre escrita pelo usuário
data_cadastro	data		sim	Data de entrada no sistema

1.3.2 serie

Representa a obra audiovisual seriada.

Atributo	Tipo	Chave	Obrig.	Descrição
id_serie	inteiro	PK	sim	Identificador interno
titulo	texto(150)		sim	Título de exibição
titulo_original	texto(150)		não	Título no idioma de origem
sinopse	texto		não	Resumo da obra
ano_estreia	inteiro		não	Ano da primeira exibição
situacao	texto(15)		sim	em_exibicao , encerrada ou cancelada
pais_origem	texto(2)		não	Código de país de duas letras

1.3.3 temporada – entidade fraca

Agrupar episódios dentro de uma série. É uma **entidade fraca**: seu número não identifica nada sozinho – “temporada 2” só faz sentido quando se sabe de qual série se trata. A identificação depende, portanto, do relacionamento identificador com **serie**.

Atributo	Tipo	Chave	Obrig.	Descrição
numero	inteiro	PP	sim	Número da temporada dentro da série; maior que zero
titulo	texto(100)		não	Nome da temporada, quando houver
data_estreia	data		não	Data de estreia da temporada

1.3.4 episodio – entidade fraca

Unidade mínima de conteúdo e alvo mais frequente de registro e avaliação. Também é entidade fraca, dependente de **temporada** pelos mesmos motivos.

Atributo	Tipo	Chave	Obrig.	Descrição
numero	inteiro	PP	sim	Número do episódio dentro da temporada; maior que zero
titulo	texto(150)		sim	Título do episódio
duracao_min	inteiro		não	Duração em minutos; maior que zero
data_exibicao	data		não	Data da exibição original
sinopse	texto		não	Resumo do episódio

1.3.5 genero

Atributo	Tipo	Chave	Obrig.	Descrição
id_genero	inteiro	PK	sim	Identificador interno
nome	texto(40)	U	sim	Nome do gênero (Drama, Comédia, ...)

1.3.6 pessoa

Profissional do meio audiovisual envolvido na produção de séries.

Atributo	Tipo	Chave	Obrig.	Descrição
id_pessoa	inteiro	PK	sim	Identificador interno
nome	texto(120)		sim	Nome da pessoa
data_nascimento	data		não	Data de nascimento
nacionalidade	texto(60)		não	Nacionalidade declarada

1.3.7 avaliacao – entidade genérica

Representa o ato de avaliar: a manifestação de um usuário sobre um item do acervo. É a **superclasse** de uma especialização detalhada na Seção 1.5.

Atributo	Tipo	Chave	Obrig.	Descrição
id_avaliacao	inteiro	PK	sim	Identificador interno
nota	decimal(2,1)		não	De 0,5 a 5,0, em múltiplos de 0,5 (RN16)
texto	texto		não	Resenha escrita pelo usuário
contem_spoiler	booleano		sim	Sinaliza conteúdo que revela enredo (RN20)
data_publicacao	data/hora		sim	Momento da publicação
tipo_alvo	texto(10)	D	sim	serie, temporada ou episodio
total_curtidas	inteiro		sim	Atributo derivado; ver Seção 1.7

1.3.8 avaliacao_serie, avaliacao_temporada, avaliacao_episodio

As três subclasses da especialização. Nenhuma delas possui atributos próprios além da ligação com o alvo correspondente: o que as distingue é *sobre o que* incidem. Cada instância de *avaliacao* corresponde a exatamente uma instância de uma das três.

1.3.9 comentario

Manifestação textual de um usuário a respeito de uma avaliação.

Atributo	Tipo	Chave	Obrig.	Descrição
id_comentario	inteiro	PK	sim	Identificador interno
texto	texto		sim	Conteúdo do comentário; não pode ser vazio
data_publicacao	data/hora		sim	Momento da publicação

1.4 Relacionamentos e cardinalidades

A Tabela 1.1 enumera os relacionamentos do modelo na notação (*mín*, *máx*), isto é, a participação mínima e máxima de cada entidade no relacionamento. Participação mínima igual a 1 indica **participação total**.

Tabela 1.1: Relacionamentos do modelo conceitual

Relacionamento	Entidades	Cardinalidade	Atributos próprios
segue	usuario / usuario	(0,N) : (0,N)	data_inicio
possui_temporada	serie / temporada	(1,N) : (1,1)	–
possui_episodio	temporada / episodio	(1,N) : (1,1)	–
classifica	serie / genero	(1,N) : (0,N)	–
participa	peessoa / serie	(0,N) : (0,N)	funcao, personagem
declara_estado	usuario / serie	(0,N) : (0,N)	estado, data_atualizacao
registra	usuario / episodio	(0,N) : (0,N)	data_assistido
escreve	usuario / avaliacao	(0,N) : (1,1)	–
curte	usuario / avaliacao	(0,N) : (0,N)	data_curtida
recebe	avaliacao / comentario	(0,N) : (1,1)	–
comenta	usuario / comentario	(0,N) : (1,1)	–
responde	comentario / comentario	(0,1) : (0,N)	–

Três relacionamentos merecem comentário.

O **segue** é um **auto-relacionamento**: as duas pontas são o mesmo conjunto de entidades, e por isso exigem **papéis** distintos – *seguidor* e *seguido*. Sem os papéis, o diagrama seria ambíguo, pois não haveria como saber qual das pontas representa quem segue.

O **registra** não possui restrição de unicidade por par usuário-episódio, o que é incomum em relacionamentos desse tipo. A escolha é deliberada e implementa a RN11: como cada registro representa um evento de exibição datado, registrar o mesmo episódio duas vezes representa uma reexibição, não uma duplicação indevida. Isso faz com que **registra** se comporte, na prática, como um relacionamento com histórico.

O **responde** é um segundo auto-relacionamento, desta vez de grau 1:N: um comentário tem no máximo um comentário-pai e pode ter vários comentários-filho. A participação mínima zero em ambas as pontas reflete que comentários de primeiro nível não têm pai, e comentários sem resposta não têm filhos.

1.5 Generalização e especialização

A entidade *avaliacao* é especializada em *avaliacao_serie*, *avaliacao_temporada* e *avaliacao_episodio*. A especialização é **total** e **disjunta**:

- **Total**, porque toda avaliação incide obrigatoriamente sobre algum alvo (RN15). Não existe instância de *avaliacao* que não pertença a uma das três subclasses.
- **Disjunta**, porque os alvos são mutuamente exclusivos: uma mesma avaliação não pode incidir simultaneamente sobre uma série e sobre um episódio. Avaliar a série e avaliar um de seus episódios são dois atos distintos, que geram duas avaliações.

O atributo *tipo_alvo* atua como **discriminador**, indicando a qual subclasse cada instância pertence.

***Orientação:** Esta é a decisão de projeto mais importante do capítulo, e deve ser retomada na Seção 2.1 quando o mapeamento for feito. A alternativa descartada é descrita abaixo*

– vale reproduzir esse raciocínio na apresentação, porque demonstra que a escolha foi consciente.

A alternativa considerada foi tratar **avaliacao** como entidade única, com três atributos de ligação opcionais, dos quais exatamente um estaria preenchido. Essa solução é comum em sistemas reais, porque simplifica as consultas: todas as avaliações ficam em uma só tabela. Ela foi descartada aqui por duas razões. A primeira é semântica: uma entidade em que a maior parte dos atributos de ligação é obrigatoriamente nula não expressa a realidade do minimundo, expressa uma acomodação de implementação. A segunda é de integridade: garantir que exatamente um dos três atributos esteja preenchido exige uma restrição composta que o modelo conceitual não consegue enunciar, apenas comentar. Com a especialização, a exclusividade é uma propriedade *estrutural* do modelo, e não uma regra externa a ele.

1.6 Agregação

A entidade **avaliacao** é, conceitualmente, o resultado de uma associação entre um usuário e um item do acervo – ou seja, é um **relacionamento reificado**. O modelo a trata como uma **agregação**, e isso tem uma consequência concreta: os relacionamentos **curte** e **recebe** não ligam usuários a séries nem usuários a usuários; eles ligam usuários à *associação inteira*.

A distinção é observável no comportamento do sistema. Quando um usuário **curte** uma avaliação, ele não está curtindo a série (o que seria um relacionamento entre **usuario** e **serie**) nem o autor (o que seria um relacionamento entre **usuario** e **usuario**). Ele está curtindo o fato de que *aquela pessoa específica avaliou aquele item específico daquele modo*. Se a avaliação for excluída, a curtida perde o objeto e deixa de existir (RN24) – o que não aconteceria se ela estivesse ligada apenas à série ou apenas ao autor.

É por isso que, no diagrama, **curte** e **recebe** partem do retângulo que envolve a agregação, e não de uma das entidades participantes isoladamente.

1.7 Atributos derivados

O atributo **total_curtidas** de **avaliacao** é **derivado**: seu valor é obtido pela contagem das instâncias de **curte** associadas àquela avaliação. Atributos derivados costumam ser omitidos do modelo, justamente por serem redundantes.

Optou-se por mantê-lo por uma razão prática: a ordenação de avaliações por popularidade é uma das operações mais frequentes do sistema, e recalculá-lo a cada leitura é caro. A redundância é aceita conscientemente, mas exige contrapartida – um mecanismo que garanta que o valor armazenado jamais divirja da contagem real. Esse mecanismo é especificado na Seção 3.4, na forma de um gatilho. A decisão ilustra um ponto recorrente em projeto de banco de dados: desnormalizar é legítimo, desde que a consistência seja garantida por construção e não por disciplina de quem escreve a aplicação.

1.8 Diagrama entidade-relacionamento

***Orientação:** O diagrama deve ser produzido no **brModelo**, em notação de Chen, exportado como imagem e salvo em **images/der-conceitual.png**. Depois, basta descomentar o bloco de figura abaixo. O uso de uma ferramenta CASE atende ao item 7 do conteúdo programático da disciplina. Confirmam, ao desenhar: (i) entidades fracas com retângulo duplo e relacionamento identificador com losango duplo; (ii) papéis nomeados nos dois auto-relacionamentos; (iii) triângulo de especialização com as marcações de totalidade*

e disjunção; (iv) retângulo tracejado envolvendo a agregação; (v) atributo derivado com elipse tracejada.

A especificação textual das Seções 1.3 a 1.7 é suficiente e não ambígua para a construção do diagrama: toda entidade, todo atributo com seu tipo e caráter de chave, toda cardinalidade e toda construção especial (entidade fraca, especialização, agregação, atributo derivado) está enunciada. O arquivo `diagramas/der-conceitual.dot` contém um rascunho estrutural que pode ser usado para conferência, mas não substitui o diagrama em notação de Chen.

Capítulo 2

Modelagem Lógica

Orientação: Capítulo a ser escrito. Ele converte o modelo conceitual em um esquema relacional e demonstra que o resultado está normalizado. A ordem das seções corresponde à ordem em que o raciocínio deve ser apresentado: primeiro as regras de conversão, depois o esquema obtido, depois a verificação de que ele é bom.

2.1 Regras de mapeamento aplicadas

Orientação: Enunciem cada regra de conversão do modelo ER para o relacional e mostrem onde ela foi aplicada. Não basta apresentar o esquema pronto: o professor avalia o processo de mapeamento (item 6.1 do conteúdo). A tabela abaixo já está organizada por tipo de construção; preencham a coluna da direita.

Tabela 2.1: Regras de mapeamento do modelo ER para o modelo relacional

Construção no ER	Regra aplicada	Onde ocorre
Entidade forte		usuario, serie, genero, pessoa, avaliacao, comentario
Entidade fraca		temporada, episodio
Relacionamento 1:N		possui_temporada, possui_episodio, escreve, recebe, comenta, responde
Relacionamento N:N sem atributo		classifica
Relacionamento N:N com atributo		participa, declara_estado, registra, curte
Auto-relacionamento N:N		segue
Auto-relacionamento 1:N		responde
Especialização total e disjunta		avaliacao e suas três subclasses
Atributo derivado		total_curtidas

2.1.1 Tratamento da especialização

Orientação: Esta subseção é obrigatória. Existem três estratégias clássicas para mapear especialização: (a) tabela única com atributos nulos e discriminador; (b) uma tabela por subclasse, ligada à superclasse; (c) uma tabela por subclasse concreta, sem tabela para a superclasse. Apresentem as três, digam qual foi adotada e por quê. A estratégia escolhida

no projeto é a (b) – justifiquem-na considerando que (i) a superclasse possui atributos próprios relevantes (*nota*, *texto*, *data_publicacao*), (ii) curtidas e comentários precisam referenciar a avaliação independentemente do tipo de alvo, o que exige uma chave única compartilhada, e (iii) a estratégia (c) tornaria impossível a referência da agregação descrita na Seção 1.6.

2.1.2 Tratamento das entidades fracas

Orientação: Expliquem a diferença entre a chave da entidade fraca no modelo conceitual (chave parcial + chave do proprietário) e a solução adotada no esquema físico. No projeto, adotou-se chave primária substituta (*id_temporada*, *id_episodio*) acompanhada de restrição de unicidade sobre o par (proprietário, número). Justifiquem: a chave composta natural funcionaria, mas se propagaria para todas as tabelas que referenciam episódio – *registro* e *avaliacao_episodio* – aumentando o tamanho das chaves estrangeiras sem ganho semântico. A restrição **UNIQUE** preserva integralmente a regra original (RN06 e RN07).

2.2 Esquema relacional

Orientação: Apresentem o esquema na notação $R(\underline{\text{chave}}, \text{atributo}, \dots)$, com chaves estrangeiras indicadas. São 17 relações. Os nomes de tabela e de coluna estão fixados no Capítulo 1 e não devem ser alterados sem atualizar aquele capítulo. A lista abaixo enumera as relações esperadas, na ordem em que devem ser apresentadas.

1. *usuario*
2. *segue*
3. *serie*
4. *temporada*
5. *episodio*
6. *genero*
7. *serie_genero*
8. *pessoa*
9. *participacao*
10. *estado_serie*
11. *registro*
12. *avaliacao*
13. *avaliacao_serie*
14. *avaliacao_temporada*
15. *avaliacao_episodio*
16. *curtida*
17. *comentario*

2.3 Dependências funcionais

Orientação: Para cada relação, listem as dependências funcionais na forma $X \rightarrow Y$ e identifiquem as chaves candidatas. Esta seção é o insumo da seção seguinte: sem as dependências explícitas, a demonstração de normalização fica sem base. A tabela abaixo já traz as relações; preencham as duas colunas restantes. Atenção especial às relações de chave composta – *participacao*, *estado_serie*, *serie_genero*, *segue* e *curtida* – que são onde uma violação de 2FN apareceria, caso existisse.

Relação	Dependências funcionais	Chaves candidatas
usuario		
segue		
serie		
temporada		
episodio		
genero		
serie_genero		
pessoa		
participacao		
estado_serie		
registro		
avaliacao		
avaliacao_serie		
avaliacao_temporada		
avaliacao_episodio		
curtida		
comentario		

2.4 Normalização

Orientação: O esquema apresentado na Seção 2.2 já nasce na 3FN, porque foi obtido por mapeamento de um modelo ER bem construído. Isso cria um problema de apresentação: não há nada para normalizar, e a seção corre o risco de virar a afirmação vazia “está tudo normalizado”.

A forma recomendada de resolver isso é **partir de uma relação universal propositalmente mal projetada** e decompô-la passo a passo até chegar ao esquema final, mostrando em cada passo qual anomalia é eliminada. Sugestão de relação de partida, que concentra os três problemas clássicos:

AVALIACAO_COMPLETA(id_usuario, id_episodio, username, email, titulo_serie, ano_estreia, numero_temporada, numero_episodio, titulo_episodio, duracao_min, nota, texto, data_publicacao, generos)

Roteiro sugerido:

- **1FN** – o atributo **generos** é multivalorado (uma série tem vários gêneros). Mostrem a anomalia concreta: impossibilidade de consultar “séries de drama” sem varrer **texto**. A decomposição gera **genero** e **serie_genero**.
- **2FN** – com chave composta (id_usuario, id_episodio), os atributos **username** e **email** dependem apenas de id_usuario, e **titulo_episodio** e **duracao_min** dependem apenas de id_episodio: são dependências parciais. Mostrem a anomalia de atualização (trocar o e-mail exige alterar todas as avaliações do usuário) e a de exclusão (apagar a última avaliação apagaria o usuário). A decomposição gera **usuario** e **episodio**.
- **3FN** – **titulo_serie** e **ano_estreia** dependem de **numero_temporada**, que por sua vez depende do episódio: dependência transitiva. A decomposição gera **serie** e **temporada**, completando a hierarquia.

Fechem a seção afirmando que o esquema final está em 3FN e verificando, relação a relação, que nenhum atributo não-primo depende parcial ou transitivamente da chave. Se decidirem discutir BCNF, verifiquem que todo determinante é chave candidata – no esquema

proposto isso se verifica, pois não há relações com duas chaves candidatas sobrepostas.

2.5 Diagrama lógico

Orientação: Diagrama do esquema relacional, com todas as tabelas, colunas, chaves primárias e chaves estrangeiras – notação de “pé de galinha” ou equivalente. Pode ser gerado no brModelo (a partir do conceitual) ou por engenharia reversa a partir do banco criado, usando pgAdmin ou DBeaver. Salvem em `images/der-logico.png` e descomentem o bloco abaixo. Se usarem engenharia reversa, confirmem se o diagrama não ficou incompleto: ferramentas costumam omitir restrições *CHECK* e gatilhos, que então precisam ser mencionados no texto.

Capítulo 3

Modelagem Física

***Orientação:** Capítulo a ser escrito. Contém o script SQL executável no PostgreSQL e sua explicação. O script completo vai no apêndice ou em arquivo anexo; neste capítulo entram os trechos que ilustram decisões, sempre acompanhados de explicação. Código sem texto explicativo não pontua.*

3.1 Convenções de nomenclatura e criação do esquema

***Orientação:** Registrem as convenções adotadas – elas já estão em uso no Capítulo 1 e devem ser seguidas: nomes de tabela no singular e em minúsculas; chave primária no formato `id_<tabela>`; `snake_case` para colunas compostas; objetos criados dentro de um esquema próprio, e não em `public`.*

```
DROP SCHEMA IF EXISTS serie_tracker CASCADE;  
CREATE SCHEMA serie_tracker;  
SET search_path TO serie_tracker;
```

Listing 3.1: Criação do esquema

3.2 Definição de dados (DDL)

***Orientação:** Apresentem os comandos `CREATE TABLE` na ordem correta de dependência: uma tabela só pode ser criada depois daquelas que ela referencia. A ordem da lista da Seção 2.2 já respeita isso e pode ser seguida diretamente. Não é necessário reproduzir as 17 tabelas no corpo do texto; selecionem as mais representativas – sugestão: `episodio` (entidade fraca com `UNIQUE` sobre a chave parcial), `participacao` (chave composta e `CHECK` cruzado entre colunas) e `avaliacao` com uma de suas subclasses (especialização, em que a chave primária é também chave estrangeira).*

3.3 Restrições de integridade

***Orientação:** Organizem por tipo, seguindo o item 5 do conteúdo programático, e citem a regra de negócio correspondente ao lado de cada restrição:*

- **Restrições de domínio** – `NOT NULL`, `CHECK`, `UNIQUE`. Casos a destacar: faixa e granularidade da nota (RN16); avaliação não vazia (RN17); data de exibição não futura (RN12); `funcao` e `personagem` coerentes entre si (RN10); proibição de seguir a si mesmo (RN03).

- *Integridade referencial* – justifiquem a política de cada chave estrangeira. Use-se *ON DELETE CASCADE* onde há dependência existencial (RN04, RN08, RN24) e *ON DELETE RESTRICT* onde a exclusão deve ser impedida por representar perda de informação do acervo.

3.4 Gatilhos

Orientação: Cinco gatilhos estão previstos. Para cada um, apresentem o código e – mais importante – expliquem **por que um gatilho é necessário**, isto é, por que uma restrição declarativa não resolveria. Esse é o argumento que diferencia um trabalho bom de um trabalho apenas correto.

Tabela 3.1: Gatilhos a implementar

Gatilho	Evento	Regra garantida
<code>trg_avaliacao_exige_registro</code>	BEFORE INSERT em <code>avaliacao_episodio</code>	RN18 – só se avalia episódio já registrado como assistido. Exige consulta a outra tabela, o que um CHECK não pode fazer.
<code>trg_avaliacao_unica_por_alvo</code>	BEFORE INSERT nas três subclasses	RN19 – uma avaliação por usuário por alvo. O autor está na superclasse e o alvo na subclasse, de modo que nenhuma restrição UNIQUE alcança as duas colunas simultaneamente.
<code>trg_bloqueia_autocurtida</code>	BEFORE INSERT em <code>curtida</code>	RN22 – ninguém curte a própria avaliação. A comparação envolve uma coluna de outra tabela.
<code>trg_atualiza_contador_curtidas</code>	AFTER INSERT/DELETE em <code>curtida</code>	Mantém o atributo derivado total_curtidas (Seção 1.7) sincronizado com a contagem real.
<code>trg_garante_estado_assistindo</code>	AFTER INSERT em <code>registro</code>	RN14 – registrar um episódio promove a série a <i>assistindo</i> . Regra de propagação de estado, inexprimível de forma declarativa.

3.5 Visões

Orientação: *Duas visões estão previstas (item 4.6 do conteúdo programático). Apresentem o código e o problema que cada uma resolve.*

- *`vw_avaliacao_completa` – reúne a superclasse e as três subclasses em um único conjunto de linhas, resolvendo o título do alvo (nome da série, identificação da temporada ou do episódio). Implementada com **UNION ALL**, exercita operações de conjunto e resolve, do lado da leitura, a fragmentação introduzida pela especialização. É a visão*

que as consultas da seção seguinte devem usar.

- `vw_progresso_serie` – para cada par usuário-série, apresenta episódios distintos assistidos, total de episódios e percentual de progresso. Atenção: use-se `COUNT(DISTINCT id_episodio)`, pois a RN11 permite registros repetidos do mesmo episódio e a contagem simples inflaria o progresso.

3.6 Carga de dados (DML)

Orientação: A carga precisa ser **coerente e suficiente**: coerente no sentido de respeitar todas as restrições e gatilhos (por exemplo, não tente inserir avaliação de episódio sem inserir antes o registro correspondente, ou a carga falhará por causa da RN18); suficiente no sentido de permitir que as três consultas da próxima seção retornem resultados não triviais.

Dimensionamento sugerido: 6 a 8 usuários, 5 a 6 séries com 2 a 3 temporadas cada, 6 a 10 episódios por temporada, 8 a 10 gêneros, 10 a 15 pessoas com participações variadas, um grafo de seguidores com pelo menos um caso de relação não recíproca, 40 ou mais registros de exibição, 20 ou mais avaliações distribuídas entre os três níveis, e curtidas e comentários suficientes para que a ordenação por popularidade produza um ranking com empates desfeitos.

Ordem de inserção: a mesma da criação das tabelas. Insiram os registros de exibição antes das avaliações de episódio.

3.7 Consultas de exemplo

Orientação: Três consultas que demonstrem a utilidade do banco no minimundo. Para cada uma: enunciem em português a pergunta respondida, apresentem o SQL, mostrem o resultado obtido e comentem-no. As três abaixo foram escolhidas para cobrir itens distintos do conteúdo programático – não as substituam por variações do mesmo tipo de consulta.

3.7.1 Consulta 1 – Ranking de séries por nota média

Orientação: Quais séries têm as melhores avaliações entre os usuários, considerando apenas as que receberam número significativo de notas?

Exercita junção múltipla, funções agregadas (`AVG`, `COUNT`), `GROUP BY` e `HAVING`. O `HAVING COUNT(*) >= 3` é o ponto central: sem ele, uma série com uma única nota 5,0 lideraria o ranking, o que evidencia por que a cláusula existe. Comentem isso.

3.7.2 Consulta 2 – Atividade recente de quem o usuário segue

Orientação: Quais foram as últimas avaliações publicadas pelos usuários que determinado usuário segue?

Exercita subconsulta aninhada (`IN` ou `EXISTS` sobre `segue`), auto-relacionamento e uso de visão (`vw_avalicao_completa`). Vale apresentar as duas formulações – com `IN` e com `EXISTS` – e comentar a diferença.

3.7.3 Consulta 3 – Séries na lista de interesse ainda não iniciadas

Orientação: Quais séries o usuário marcou como “pretendo assistir” sem ter assistido nenhum episódio até agora?

Exercita operação de conjunto (EXCEPT), que é o item 4.3 do conteúdo. É a consulta que sustenta a funcionalidade real de “comece por aqui”. Vale mostrar também a formulação equivalente com NOT EXISTS e comentar que ambas retornam o mesmo conjunto.

3.8 Rastreabilidade das regras de negócio

Orientação: Tabela de fechamento do trabalho: para cada regra da Seção 1.2, indiquem onde ela foi implementada. Regras sem implementação precisam ser justificadas – normalmente porque pertencem à camada da aplicação. Esta tabela é a melhor defesa possível numa arguição, porque mostra que nada foi enunciado e esquecido.

Regra	Implementada por	Observações
RN01		
RN02		
RN03		
RN04		
RN05		
RN06		
RN07		
RN08		
RN09		
RN10		
RN11		
RN12		
RN13		
RN14		
RN15		
RN16		
RN17		
RN18		
RN19		
RN20		
RN21		
RN22		
RN23		
RN24		

Apêndice A

Divisão de trabalho e convenções do grupo

Este apêndice é material interno e não faz parte da entrega. Ele desaparece do PDF quando `\orientacoesfalse` é acionado.

A.1 Atribuições

Integrante	Parte	Entrega
Igor	Capítulo 1	Concluído: minimundo, 24 regras de negócio, dicionário de dados, cardinalidades, especialização, agregação. Falta apenas desenhar o DER no brModelo.
–	Capítulo 2	Mapeamento ER → relacional, esquema relacional, dependências funcionais, demonstração de normalização e diagrama lógico.
–	Capítulo 3, seções 1–5	DDL das 17 tabelas, restrições de integridade, os cinco gatilhos e as duas visões.
–	Capítulo 3, seções 6–8	Carga de dados, as três consultas com resultados comentados e a tabela de rastreabilidade. Responsável também por executar o script inteiro do zero e confirmar que roda sem erro.

A.2 Contrato de nomes

O Capítulo 1 é a fonte de verdade para nomes de tabelas, colunas, tipos e domínios. Ninguém deve renomear nada durante a escrita dos capítulos seguintes sem atualizar o capítulo conceitual na mesma edição. O erro mais comum neste tipo de trabalho é o esquema físico divergir do diagrama conceitual – e ele é penalizado porque torna o documento internamente contraditório.

Em caso de necessidade real de mudança, anunciem ao grupo antes de alterar.

A.3 Verificação antes da entrega

1. O script SQL executa do zero, em banco vazio, sem erro e sem advertência.
2. Toda regra de negócio da Seção 1.2 aparece preenchida na tabela de rastreabilidade.
3. Os nomes no DDL coincidem com os do dicionário de dados.
4. As duas figuras de diagrama estão presentes e com os blocos descomentados.
5. Os integrantes e RAs na capa estão corretos.
6. `\orientacoestrue` foi trocado por `\orientacoesfalse`.

7. O sumário foi regenerado (compilar duas vezes).